

1 Introduction

Global Navigation Satellite Systems (GNSS), which include GPS, GLONASS, Galileo, and BeiDou, are essential for providing precise positioning, navigation, and timing data. However, GNSS signals face challenges that can degrade their accuracy and reliability. Sources of error for these signals include satellite geometry, receiver clock errors, signal noise, and multipath, among others.

In today's world, where systems are increasingly complex and data is often noisy or incomplete, the ability to make accurate predictions and estimations is essential. This is where Kalman filtering plays a critical role.

Kalman filtering is an algorithm that uses a series of measurements observed over time, including statistical noise and other inaccuracies, to produce estimates of unknown variables. These estimates tend to be more accurate than those based on a single measurement because the algorithm calculates the joint probability distribution of the variables at each time step.

The Kalman filter is an optimal recursive data processing algorithm. It is designed to estimate the internal state of a dynamic system from a series of noisy measurements. It smooths out noise and predicts future states, making it invaluable for real-time applications such as navigation, tracking, and control systems.

The Kalman filter can fuse information from multiple sources, account for uncertainty, and continuously update its estimates as new data becomes available. This makes it especially useful in environments where precision and adaptability are essential. [3]

Kalman filtering provides a mathematical framework to fuse GNSS data with other sensors (e.g., accelerometers, gyroscopes) to improve accuracy, and at the same time it allows to predict and correct the system's state in real time, even when GNSS signals are temporarily lost or degraded. Kalman filtering reduce the impact of noise and measurement errors by optimally estimating the true position and velocity.

By continuously updating its estimates as new data becomes available, the Kalman filter ensures robust and reliable navigation, which is especially vital in safety-critical applications like aviation, autonomous driving, and maritime navigation, as we will see in this assignment.

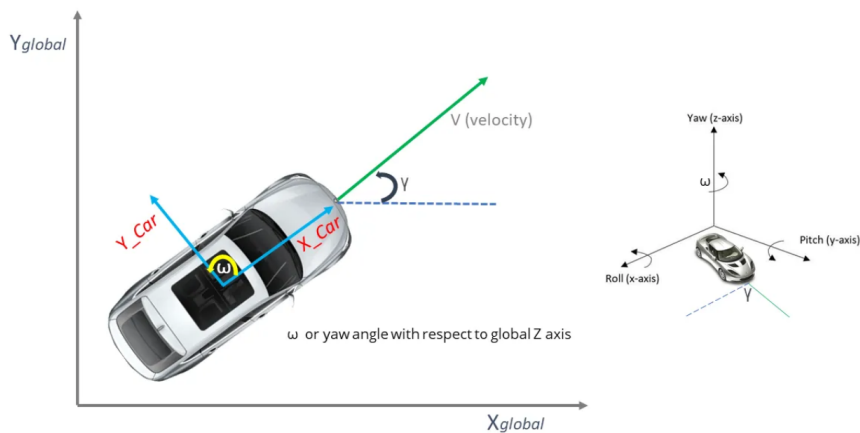


Figure 1: State estimation using Kalman Filter [1].

This report primarily aims to implement and analyze the performance of a Kalman filter in estimating the two-dimensional (2D) position and velocity of a moving vehicle. To do this, we will use noisy Global Navigation Satellite System (GNSS) like measurements. The vehicle's motion is modeled using a kinematic model with constant velocity, with measurements provided at regular 2-second intervals.

A kinematic model is a mathematical representation of an object's motion. It ignores the forces affecting the object and focuses on the geometric relationships among its components. Real-time kinematic (RTK) models estimate the position and velocity of a moving object in real time. In the context of GNSS and Kalman filtering, RTK models are crucial for forecasting a system's (e.g., a vehicle) state evolution over time, particularly when noisy measurements are present.

In this assignment, the Kalman filter computes real-time, and post-processed (smoothed) estimates of the vehicle's state, including its position and velocity components. We will evaluate the accuracy of these estimates by comparing them to the actual state values. In addition we will see how changes in key parameters affect the filter's performance.

The study emphasizes the critical role of Kalman filtering in GNSS applications, where managing measurement noise and system dynamics is essential for achieving reliable and accurate navigation solutions.

2 Methodology

As previously stated, the goal of the assignment is to apply the Kalman filter to compute the optimal two-dimensional (2D) position and velocity of a vehicle using the constant velocity kinematic model for vehicle motion.

The Kalman filter will be applied to the given vehicle motion data by following the steps explained in [2].

- **Step 1: Initialization**

We extract the data from the Measurements sheet in the Excel file *Measeurements.xlsx*. The sheet *Measurements* contains information about the position given by the ENU coordinates e and n in meters and the speed in meters per second. For the initial moment of time, we also have knowledge of the speed of the e and n components. This means that we can define the state vector at the initial time.

The state vector contains position and velocity information: $\mathbf{x} = [e \quad n \quad v_e \quad v_n]^T$.

We also define the system dynamic matrix,

$$\mathbf{F} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix},$$

and the distribution matrix

$$\mathbf{G} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

These two matrices are essential for building the matrix differential equation,

$$\dot{\mathbf{x}} = \mathbf{F}(t)\mathbf{x}(t) + \mathbf{G}(t)\mathbf{u}(t)$$

, which describes a dynamic process, such as the movement of a vehicle. The vector $\mathbf{u}$ introduces white noise.

In this first step, we define the transition matrix between the time epochs as,

$$\mathbf{T}_{k-1,k} = \mathbf{I} + \mathbf{F}_k \Delta t = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

In this case, the time interval between the epochs is 2 seconds, and the system dynamic matrix is represented by $\mathbf{F}_k$.

Given the standard deviations of the measured coordinates ($\sigma_e = 3m$, $\sigma_n = 3m$) and the measured absolute velocity ($\sigma_v = 0.5m/s$), we can compute the covariance matrix of the observations.

$$\mathbf{R} = \begin{bmatrix} \sigma_e^2 & 0 & 0 \\ 0 & \sigma_n^2 & 0 \\ 0 & 0 & \sigma_v^2 \end{bmatrix}.$$

At the same time, we can define the covariance matrix of the initial state vector as follows:

$$\mathbf{Q}_{x,0} = \begin{bmatrix} \sigma_{0,e}^2 & 0 & 0 & 0 \\ 0 & \sigma_{0,n}^2 & 0 & 0 \\ 0 & 0 & \sigma_{0,v_e}^2 & 0 \\ 0 & 0 & 0 & \sigma_{0,v_n}^2 \end{bmatrix},$$

where $\sigma_{0,e} = \sigma_{0,n} = 3$ m/s and $\sigma_{0,v_e} = \sigma_{0,v_n} = 10$ m are respectively the standard deviation of initial coordinates e and n, and standard deviation of initial velocity for both components.

The process noise covariance matrix can be defined as

$$\mathbf{Q}_k = \mathbf{Q}_G \Delta t + (\mathbf{F} \mathbf{Q}_G + \mathbf{Q}_G \mathbf{F}^T) \frac{\Delta t^2}{2} + \mathbf{F} \mathbf{Q}_G \mathbf{F}^T \frac{\Delta t^3}{3} = \begin{bmatrix} \frac{q_e \Delta t^3}{3} & 0 & \frac{q_e \Delta t^2}{2} & 0 \\ 0 & \frac{q_n \Delta t^3}{3} & 0 & \frac{q_n \Delta t^2}{2} \\ \frac{q_e \Delta t^2}{2} & 0 & q_e \Delta t & 0 \\ 0 & \frac{q_n \Delta t^2}{2} & 0 & q_n \Delta t \end{bmatrix},$$

where $\mathbf{F}$ is the system dynamic matrix and Δt is again the time interval. We define the matrix $\mathbf{Q}_G$ as $\mathbf{Q}_G = \mathbf{G} \mathbf{Q} \mathbf{G}^T$, where $\mathbf{G}$ is the distribution matrix and the matrix $\mathbf{Q}$ is the noise covariance matrix,

$$\mathbf{Q} = \begin{bmatrix} q_{ae} & 0 \\ 0 & q_{an} \end{bmatrix}.$$

Here q_{ae} and q_{an} are the power spectral density (PSD) of the random acceleration, that in this case takes a value of $0.01 \text{ m}^2\text{s}^{-3}$.

• Step 2: Time propagation

In the second step, we can compute the predicted state vector as

$$\mathbf{x}_k^- = \mathbf{T}_{k-1,k} \mathbf{x}_{k-1},$$

where $\mathbf{T}_{k-1,k}$ is the transition matrix from the previous epoch to the actual one, this matrix have the same structure the one explained in the previous step. And $\mathbf{x}_{k-1}$ is the state vector in the previous epoch.

In this step, we also compute the predicted covariance matrix as

$$\mathbf{Q}_{x,k}^- = \mathbf{T}_{k-1,k} \mathbf{Q}_{x,k-1} \mathbf{T}_{k-1,k}^T + \mathbf{Q}_k,$$

where $\mathbf{T}_{k-1,k}$ s the transition matrix from the previous epoch to the actual one, $\mathbf{Q}_{x,k-1}$ is the covariance matrix of the state vector and $\mathbf{Q}_k$ is the process noise covariance matrix.

• Step 3: Gain calculation

In this step we include some intermediate calculations necessary for the final updates.

From the predicted state vector we compute the predicted mean velocity value as

$$v_k^- = \sqrt{\left(v_{e,k}^-\right)^2 + \left(v_{n,k}^-\right)^2}$$

The observation matrix $\mathbf{H}_k$ is the Jacobian of the observation function evaluated at the predicted state. It contains the partial derivatives of the observation function $\mathbf{h}_k$ with respect to the state variables, it is used to linearize the non-linear measurement model.

$$\mathbf{H}_k = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \frac{v_{e,k}^-}{v_k^-} & \frac{v_{n,k}^-}{v_k^-} \end{bmatrix}$$

From the matrices and vectors computed until now, we can compute the Kalman gain matrix, $\mathbf{K}_k$, that determines how much the prediction should be corrected based on the measurement residual.

$$\mathbf{K}_k = \mathbf{Q}_{x,k}^- \mathbf{H}_k^T \left[\mathbf{R}_k + \mathbf{H}_k \mathbf{Q}_{x,k}^- \mathbf{H}_k^T \right]^{-1}$$

- **Step 4: Measurement update**

The predicted mean velocity value (v_k^- computed in the previous step is included in the predicted observation vector $\mathbf{h}_k = [e_k^- \ n_k^- \ v_k^-]^T$. This vector is obtained by mapping the predicted state vector to the measurement space using the nonlinear observation function.

The observation vector $\mathbf{L}_k$ contains the observed east and north coordinates, and the absolute velocity (speed) at epoch k, given in the Excel file.

With this new vectors, Kalman gain matrix, $\mathbf{K}_k$, and the predicted state vector, $\mathbf{x}_k^-$, previously computed; we can obtain the filtered state vector as

$$\mathbf{x}_k = \mathbf{x}_k^- + \mathbf{K}_k [\mathbf{L}_k - \mathbf{h}_k^-] .$$

- **Step 5: Covariance update**

We can finally compute the filtered covariance of the state vector as

$$\mathbf{Q}_{x,k} = [\mathbf{I} - \mathbf{K}_k \mathbf{H}_k] \mathbf{Q}_{x,k}^- ,$$

where is the $\mathbf{K}_k$ Kalman gain matrix, $\mathbf{H}_k$ is the observation matrix and $\mathbf{Q}_{x,k}^-$ is the predicted covariance matrix of the state vector.

While the Kalman filter provides real-time estimates of the system state by incorporating information up to the current epoch, it does not utilize future measurements. To improve the accuracy of the estimated trajectory and velocity, smoothing techniques are employed. The smoothing process implemented performs a backward pass after filtering to update each prior state estimate using future information. This leads to a significant reduction in estimation error, particularly in dynamic systems with measurement noise.

$$\mathbf{D}_k = \mathbf{Q}_k \mathbf{T}^\top (\mathbf{Q}_{k+1}^-)^{-1}$$

$$\hat{\mathbf{x}}_k = \mathbf{x}_k + \mathbf{D}_k (\hat{\mathbf{x}}_{k+1} - \mathbf{x}_{k+1}^-)$$

$$\hat{\mathbf{Q}}_k = \mathbf{Q}_k + \mathbf{D}_k (\hat{\mathbf{Q}}_{k+1} - \mathbf{Q}_{k+1}^-) \mathbf{D}_k^\top$$

Where $\hat{\mathbf{x}}_k$ is the smoothed state vector at epoch k , $\hat{\mathbf{Q}}_k$ is smoothed covariance matrix at epoch k and $\mathbf{D}_k$ is the smoothing gain matrix.

And $\mathbf{x}_k$ is the filtered state vector at epoch k , $\mathbf{x}_{k+1}^-$ is the predicted state vector at epoch $k + 1$, $\mathbf{Q}_k$ is the filtered covariance matrix at epoch k and $\mathbf{Q}_{k+1}^-$ is the predicted covariance matrix at epoch $k + 1$. These values were computed during the application of the Kalman filter.

The smoother gain determines how much the information from future measurements should influence the current state. The recursion starts from the final filtered state and moves backward in time, refining the state and covariance estimates for all previous epochs.

We implement the Kalman filter in Matlab. First, we created a program to compute the initialization step using the given initial data. Then, we created a loop that repeats steps two through five for each measurement observation. For each iteration, we save the predicted state vector, the predicted covariance matrix, the filtered state vector, and the covariance matrix.

After implementing the Kalman filter, we use a backward loop to apply the smoothing process starting with the last filtered observation until reaching the first one.

To verify the accuracy of our results, we compare the values obtained in the first two iterations of applying the Kalman filter with those in the *IntermediateResults.pdf* file. The accuracy of our computations is evident.

We can also compute the error between the real values given in the Excel file *Measurements.xlsx* sheet *True values*, and the filtered and smoothed results, as the difference between the components in these vectors. With the help of Matlab we can plot these errors.

3 Results

The performance of the Kalman filter and the smoothing process was evaluated using measured positions and velocities of a moving vehicle, with known ground truth values. Tables 1, 2, and 3 list the predicted, filtered, and smoothed state vectors at each 2-second interval, including east and north positions, and the corresponding velocity components. We have compare the values obtained in the Table 2 with some of the intermediate results included in the *IntermediateResults.pdf* file, and we can see that the results are the same for the first iterations.

Figure 2 shows the 2D trajectory of the vehicle in the East–North coordinate system, comparing the raw measurements, Kalman filter estimates, smoothed estimates, and the true path. The measured positions (black dots) exhibit noticeable scatter due to noise in the observations. The filtered trajectory (blue line) follows the general trend of the true path but shows small deviations, especially where the trajectory changes more rapidly or where measurements are noisy.

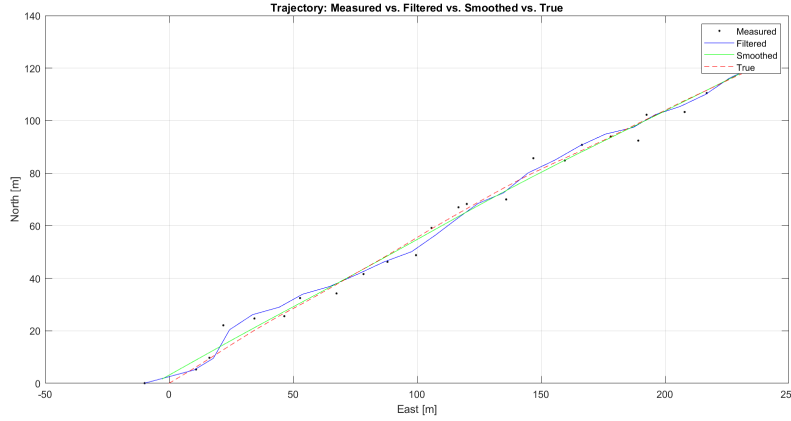


Figure 2: Trajectory comparison

The smoothed trajectory (green line), demonstrates a closer fit to the true trajectory (dashed red line) across the entire motion. It successfully suppresses high-frequency noise while retaining the correct overall shape and curvature of the path. Notably, in segments where the filter overshoots or undershoots the true path, the smoother effectively corrects these deviations, leading to a trajectory that more accurately represents the vehicle’s motion.

To complete the analysis between the computed coordinates and velocities and the real values, we have compute the error values with respect to the true values.

Figure 3 shows the position error (difference from true position) for both the filtered and smoothed estimates in the east (e) and north (n) directions. The filtered estimates fluctuate more and occasionally overshoot the ground truth, while the smoothed estimates demonstrate a consistently reduced error across the entire trajectory. For example, in the east direction, the filtered error peaks around -9 m initially, while smoothing brings this closer to -3 m. In the north direction, filtered errors briefly exceed 5 m but are also significantly reduced after smoothing.

With respect to the velocities, the figure 4 shows the velocity error (difference from true velocities) for both the filtered and smoothed estimates in the east component (v_e) and north component (v_n). As in the case of the position, we can see that there exists some fluctuation between the estimated values and the real ones. However, the smoothed values keep closer to the true ones, specially with

respect to the velocity in the north component. When we analyze the velocity in the east component, we can see that there is not much difference between the smooth and the filtered values, indicating that the Kalman filter already provides a strong estimate in this direction. Despite the smaller improvement in v_e , the smoothed estimates still demonstrate slightly better alignment with the true values, suggesting a marginal reduction in estimation variance.

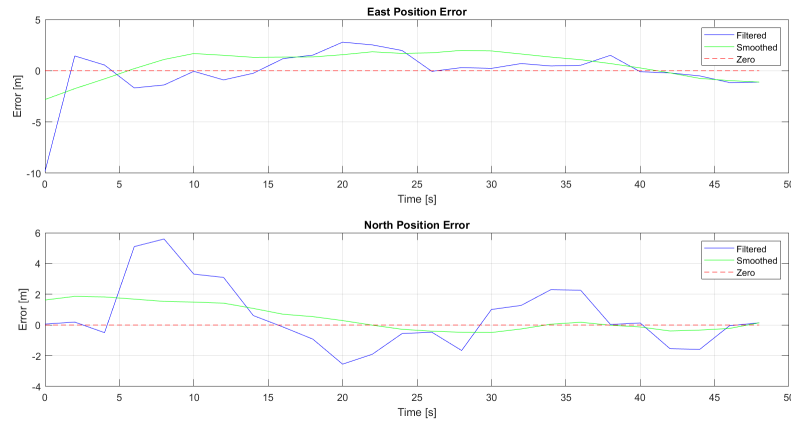


Figure 3: Position comparison

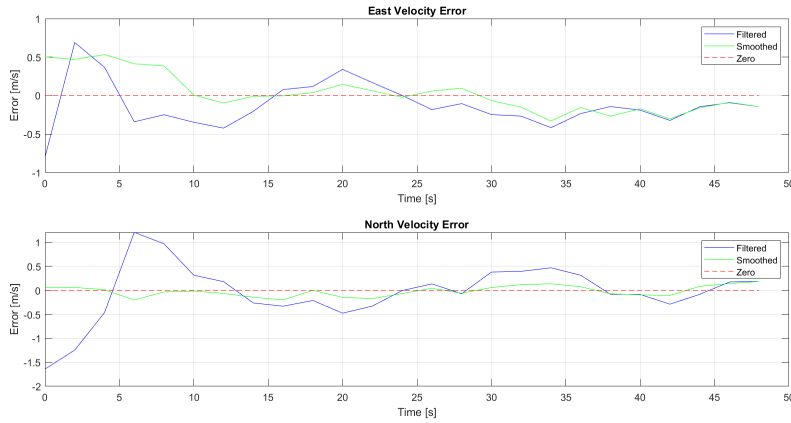


Figure 4: Velocity comparison

Time [s]	Predicted East [m]	Predicted North [m]	Predicted v_e [m/s]	Predicted v_n [m/s]
0	-9.82	0.06	3.53	0.86
2	-2.76	1.78	3.53	0.86
4	20.159	7.5709	5.0538	1.2529
6	27.264	13.7	4.6843	2.0835
8	32.719	28.325	4.1536	3.9662
10	42.37	33.246	4.3404	3.5524
12	53.75	34.717	4.6583	2.8674
14	63.267	39.496	4.7208	2.7895
16	74.511	41.762	4.8624	2.4408
18	86.656	46.622	5.1622	2.4722
20	97.005	51.074	5.1151	2.4045
22	108.28	54.749	5.2057	2.3036
24	117.49	61.287	5.0891	2.4679
26	126.86	68.499	4.9912	2.6545
28	134.16	73.804	4.7246	2.6356
30	144.39	77.451	4.7936	2.4925
32	154.44	85.652	4.8272	2.7723
34	165.33	90.291	4.8798	2.6596
36	175.44	95.742	4.9066	2.6689
38	185.71	99.953	4.8968	2.5413
40	197.43	102.02	5.1010	2.2911
42	205.85	106.88	4.9369	2.3143
44	216.08	109.73	4.9209	2.1342
46	226.33	114.36	4.9748	2.1738
48	235.82	120.91	4.9695	2.3842

Table 1: Predicted State Vectors: Coordinates and Velocities

Time [s]	Filtered East [m]	Filtered North [m]	Filtered v_e [m/s]	Filtered v_n [m/s]
0	-9.82	0.06	3.53	0.86
2	10.051	5.0651	5.0538	1.2529
4	17.895	9.5328	4.6843	2.0835
6	24.412	20.393	4.1536	3.9662
8	33.69	26.141	4.3404	3.5524
10	44.434	28.983	4.6583	2.8674
12	53.826	33.917	4.7208	2.7895
14	64.786	36.881	4.8624	2.4408
16	76.332	41.678	5.1622	2.4722
18	86.775	46.265	5.1151	2.4045
20	97.872	50.142	5.2057	2.3036
22	107.31	56.352	5.0891	2.4679
24	116.87	63.19	4.9912	2.6545
26	124.71	68.532	4.7246	2.6356
28	134.8	72.466	4.7936	2.4925
30	144.79	80.107	4.8272	2.7723
32	155.57	84.972	4.8798	2.6596
34	165.62	90.404	4.9066	2.6689
36	175.92	94.87	4.8968	2.5413
38	187.22	97.434	5.1010	2.2911
40	195.97	102.25	4.9369	2.3143
42	206.23	105.46	4.9209	2.1342
44	216.38	110.01	4.9748	2.1738
46	225.88	116.14	4.9695	2.3842
48	236.01	120.66	4.9639	2.3451

Table 2: Filtered State Vectors: Coordinates and Velocities

Time [s]	Smoothed East [m]	Smoothed North [m]	Smoothed v_e [m/s]	Smoothed v_n [m/s]
0	-2.8081	1.6247	4.8330	2.5548
2	6.86	6.7379	4.8345	2.5583
4	16.542	11.857	4.8502	2.5598
6	26.297	16.976	4.9074	2.5562
8	36.18	22.081	4.9742	2.5496
10	46.168	27.164	5.0121	2.5362
12	56.229	32.242	5.0466	2.5443
14	66.338	37.345	5.0574	2.5606
16	76.48	42.51	5.0812	2.6034
18	86.599	47.73	5.0358	2.6157
20	96.646	52.98	5.0110	2.6325
22	106.64	58.244	4.9836	2.6256
24	116.59	63.465	4.9669	2.5914
26	126.52	68.602	4.9658	2.5434
28	136.49	73.651	4.9935	2.5036
30	146.49	78.609	5.0084	2.4498
32	156.5	83.44	4.9959	2.3816
34	166.49	88.155	4.9933	2.3348
36	176.46	92.787	4.9753	2.2999
38	186.41	97.384	4.9769	2.3023
40	196.34	101.99	4.9538	2.3049
42	206.24	106.61	4.9412	2.3160
44	216.14	111.27	4.9608	2.3412
46	226.08	115.97	4.9741	2.3513
48	236.01	120.66	4.9639	2.3451

Table 3: Smoothed State Vectors: Coordinates and Velocities

To evaluate the impact of the process noise level on the estimation accuracy, the power spectral density (PSD) parameter of the random acceleration was modified from its original value of $\text{PSD} = 0.01 \text{ m}^2/\text{s}^3$ to higher values of 0.05 and $0.1 \text{ m}^2/\text{s}^3$. This adjustment increases the assumed model uncertainty, allowing the Kalman filter and the smoothing process to respond more flexibly to deviations in the measurements.

In the trajectory plot (East–North plane), it was observed that with higher PSD values, the smoothed trajectory aligns even more closely with the true trajectory, and in some sections, the smoothed and true paths nearly overlap, particularly when $\text{PSD} = 0.1$. This confirms that increasing the model noise can improve tracking performance when the system dynamics deviate slightly from the constant velocity assumption.

Additionally, in the error plots of state variables, the smoothed estimates showed improved accuracy compared to the original configuration. The residuals between the smoothed states and the true values were reduced across most state components. Notably, the east velocity component (v_e), which previously showed a slightly poorer match compared to v_n , also exhibited reduced error with the higher PSD, although some deviation remains.

These results highlight the importance of correctly tuning the process noise. A higher PSD allows the filter and smoother to adapt more aggressively to changing dynamics, which can be beneficial in real-world navigation scenarios where the constant velocity model is only an approximation.

In addition to tuning the process noise, the effect of modifying the standard deviations of the measurements was analyzed. These values determine the measurement noise covariance matrix R , which controls how much the Kalman filter and smoother trust the measurements versus the model predictions.

To evaluate this, the original standard deviations were doubled and halved, and then also reduced significantly (by multiplying by a factor of 0.01). It was observed that increasing the standard

deviations causes the smoothed solution to converge toward the filtered one. This occurs because the filter and smoother place less trust in the measurements and rely more on the predicted states from the motion model. As a result, the smoother has less ability to adjust past states, and both filtered and smoothed solutions exhibit similar behavior.

On the other hand, reducing the standard deviations (e.g., halving the values) slightly improves the smoothed solution, although the improvement is marginal. When the measurement noise is reduced drastically (e.g., by a factor of 0.01), both the filter and the smoother follow the measurements almost exactly. In this case, the estimation process does not perform meaningful correction, and the estimated states simply replicate the noisy observations — which defeats the purpose of filtering.

The trajectory plot shows minimal visual change across these scenarios, since the path shape is consistent, but the error plots and state deviations reveal that overly trusting the measurements reduces the value of smoothing. These results highlight the importance of balancing measurement trust and model confidence to achieve optimal filtering and smoothing performance.

In addition, we tested the sensitivity of the filter and the smoothing process to changes in the standard deviations of the initial state variables, including both position and velocity. While these parameters influence the initial uncertainty assigned to the state estimate, we observed that modifying them (e.g., doubling or halving the values) did not introduce any significant change in the final results. This is expected, as the Kalman filter quickly incorporates measurements and process dynamics, and the effect of the initial uncertainty diminishes after a few iterations

4 Analysis and Discussion

Kalman filtering and smoothing procedures were applied to estimate the position and velocity of a vehicle using GNSS-like measurements with known noise characteristics. The filter tracked the system's state in real time, and the smoother significantly improved the accuracy of state estimates by incorporating future observations. This improvement was particularly evident in the position estimates and north velocity component, where the smoothed results closely aligned with the true values.

Sensitivity analysis of the process noise power spectral density (PSD) revealed that increasing the PSD improved the model's ability to adapt to changes in motion. This resulted in better alignment of the smoothed trajectory with the true values. In some cases, the smoothed trajectory overlapped with the true values. Conversely, a low PSD made the system overly reliant on the model, limiting corrections from measurements and increasing errors.

Increasing the measurement noise standard deviations diminished the smoother's influence, making the smoothed and filtered results nearly identical. This indicated that the filter relied almost entirely on the motion model. When measurement noise was drastically reduced, the filter and smoother replicated the noisy observations, confirming that overly trusting measurements leads to degraded estimation quality.

Modifying the standard deviations of the initial state variables had a negligible long-term impact, which highlights the filter's ability to rapidly adapt and converge to accurate estimates as more measurements are processed.

Overall, the smoothed estimates provided lower variance and smaller residuals than the filtered estimates, which validates the importance of smoothing when post-processing trajectory data is possible.

Kalman filtering and smoothing are essential tools for GNSS-based navigation and positioning systems. GNSS measurements are often noisy, sparse, and irregular. Filtering provides a real-time, recursive estimation of position and velocity. Smoothing uses both past and future data to produce the most accurate state estimates.

This process is particularly important in GNSS applications, where accuracy is critical for safe and reliable navigation. Kalman filtering and smoothing reduce the impact of noise by modeling system dynamics and measurement uncertainty, thereby improving position and velocity estimates and ensuring robust tracking performance. The ability to adjust parameters such as process noise and measurement error enables these algorithms to adapt to different operating environments.

References

- [1] Analytics Vidhya. *State Estimation using Extended Kalman Filter (EKF) for Robotics*. [Online; accessed 13-May-2025]. 2020. URL: <https://medium.com/analytics-vidhya/state-estimation-using-extend-kalman-filter-ekf-for-robotics-c94792f3bdb8>.
- [2] Milan Horemuz. *Kalman Filtering*. 2018.
- [3] Wikipedia contributors. *Kalman filter — Wikipedia, The Free Encyclopedia*. [Online; accessed 13-May-2025]. 2025. URL: https://en.wikipedia.org/wiki/Kalman_filter.